FC Fluxo de Caixa

DOCUMENTO 16

Serviços e ambiente local

Guia dos microsserviços, matriz de compatibilidade, desempenho de consultas e laboratório local

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

lancamentos-service 3

Desempenho das consultas — lancamentos-service 9

consolidado-service 11

Desempenho das consultas do consolidado 18

Ambiente local 20

Carga do consolidado 22

FONTE: SERVICES/LANCAMENTOS-SERVICE/README.MD

lancamentos-service

Serviço de escrita do fluxo de caixa. Registra créditos e débitos, estorna lançamentos e publica o evento `LancamentoRegistrado` sem depender do serviço de consolidado.

Responsabilidade

- Fonte da verdade dos lançamentos de um comerciante.
- Não altera nem apaga lançamento: correção é um novo lançamento de estorno (tipo oposto, mesmo valor e mesma data de competência).
- Grava o lançamento e o evento na mesma transação (transactional outbox) e um relay publica no tópico SNS `fluxo-caixa-lancamentos-events`.
- Se o consolidado estiver fora, este serviço continua aceitando lançamentos. As mensagens ficam no outbox e depois no SQS.

Arquitetura interna

Hexagonal, pacote base `br.com.fluxocaixa.lancamentos`:

```
domain/                 agregado, value objects e evento — sem Spring
application/            casos de uso e ports
adapters/in/web/        API REST, Problem Details e claim do JWT
adapters/out/persistence/  JPA + Flyway
adapters/out/messaging/    publicador SNS e relay da outbox
config/                 segurança, AWS, OpenAPI e observabilidade
```

O `comercianteId` sai da claim JWT `comerciante_id` (configurável). Ele não vai no corpo nem na URL.

Endpoints

Porta local `8081`. Base `/api/v1`. Erros em `application/problem+json` (RFC 9457).

Método	Caminho	Escopo
POST	`/api/v1/lancamentos`	`lancamentos:write`
GET	`/api/v1/lancamentos/{id}`	`lancamentos:read`
GET	`/api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`	`lancamentos:read`
POST	`/api/v1/lancamentos/{id}/estornos`	`lancamentos:write`

OpenAPI: `/v3/api-docs` e `/swagger-ui.html`. No perfil `local` o Swagger é público. No perfil `aws` o springdoc fica desligado e essas rotas exigem autenticação se alguém as religar. Igual ao consolidado.

Probes: `/actuator/health/liveness` e `/actuator/health/readiness`. Métricas: `/actuator/prometheus`.

O header `Idempotency-Key` (UUID) é opcional no registro e no estorno. A primeira chamada responde `201`; a repetição da mesma chave e do mesmo conteúdo responde `200` com o mesmo recurso. A mesma chave com outro conteúdo responde `409`.

O valor sai como string decimal com duas casas (`"150.00"`), no mesmo formato do evento.

```
curl -sS -D- -X POST http://localhost:8081/api/v1/lancamentos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"tipo":"CREDITO","valor":150.00,"dataLancamento":"2026-11-10","descricao":"Venda
      balcão","categoria":"VENDAS"}'

curl -sS "http://localhost:8081/api/v1/lancamentos?data=2026-11-10&page=0&size=50" \
  -H "Authorization: Bearer $TOKEN"

curl -sS -X POST http://localhost:8081/api/v1/lancamentos/$ID/estornos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"motivo":"Cliente devolveu"}'
```

Lista paginada:

```
{
  "conteudo": [],
  "pagina": 0,
  "tamanho": 50,
  "totalElementos": 0,
  "totalPaginas": 0
}
```

Variáveis de ambiente

Variável	Uso
`SERVER_PORT`	Porta HTTP. Padrão `8081`.
`DB_URL`	JDBC do PostgreSQL do serviço. No perfil `aws` com segredo, pode ser `jdbc:postgresql://...`; o serviço prefixa `jdbc:aws-wrapper:`.
`DB_USERNAME` / `DB_PASSWORD`	Credenciais do banco. Obrigatórias sem `DB_SECRET_ARN`.
`DB_SECRET_ARN`	ARN do segredo do RDS. Só ativa o AWS JDBC Wrapper no perfil `aws`.
`DB_READER_URL`	JDBC opcional da réplica. Ausente = um único DataSource.
`OUTBOX_RETENCAO`	Quanto tempo um evento publicado permanece na outbox. Padrão `7d`.
`OUTBOX_ADIAMENTO_CIRCUIT O`	Quanto o relay espera com o circuito aberto. Padrão `30s`.
`OUTBOX_LIMPEZA_CRON`	Cron UTC da limpeza. Padrão `0 15 4 * * *`.

Variável	Uso
`AWS_REGION`	Região. Padrão `us-east-1`.
`AWS_ENDPOINT_URL`	Endpoint do LocalStack. Vazio na AWS.
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Usadas só quando `AWS_ENDPOINT_URL` está definido. Padrão `test` / `test`. Na AWS a task role entra pela cadeia padrão do SDK.
`LANCAMENTOS_TOPIC_ARN`	ARN do tópico `fluxo-caixa-lancamentos-events`.
`JWT_ISSUER_URI`	Issuer esperado do access token.
`JWT_JWK_SET_URI`	Opcional. Busca as chaves nesse URI e continua validando o issuer.
`APP_SECURITY_SCOPE_PREFIX`	Prefixo do escopo. Vazio no local; `fluxo-caixa/` na AWS.
`APP_SECURITY_COMERCIANTE_CLAIM`	Claim do comerciante. Padrão `comerciante_id`.
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP HTTP (traces e métricas). Ex.: `http://localhost:4318`.
`SPRING_PROFILES_ACTIVE`	`local` ou `aws`.

Profiles:

- `local`: detalhe do health liberado para diagnóstico.
- `aws`: `server.forward-headers-strategy=framework` para o ALB.

Threads virtuais e shutdown gracioso ficam ligados em qualquer profile.

Rastreabilidade

- Propagação W3C (`traceparent` / `tracestate`) na entrada HTTP.
- Header `X-Correlation-Id`: UUID. Se vier ausente ou inválido, o serviço gera outro, devolve no response e grava no MDC como `correlationId`.
- Log JSON de console com `traceId`, `spanId`, `correlationId` e, quando a operação tem, `comercianteId`, `lancamentoId` e `eventId`.
- O POST e o estorno gravam `trace_parent`, `trace_state` e `correlation_id` na `outbox_eventos`.
- O relay abre um span `PRODUCER` filho desse contexto e publica os message attributes `eventType`, `traceparent`, `tracestate` (se houver) e `correlationId`. O JSON do evento não muda.

Resiliência da publicação SNS

O POST não chama o SNS. O relay publica com Resilience4j, instância `publicacaoSns`:

- Circuit breaker (janela de 10, mínimo 5 chamadas, 50% de falha, 30 s aberto).
- Retry com backoff exponencial e jitter (3 tentativas, 50 ms a 400 ms).
- Time limiter de 1 s, com timeout do cliente AWS em 2 s. O rate limiter não é usado neste serviço.

- Com o circuito aberto o lote fica na outbox (`proxima_tentativa_em` adiantada por `OUTBOX_ADIAMENTO_CIRCUITO` , padrão 30 s). Nada é descartado.
- Métricas no Micrometer (`resilience4j.circuitbreaker.*` , `resilience4j.retry.*` , `resilience4j.timelimiter.*`).
- O estado do circuito aparece em `/actuator/health` (`CIRCUIT_OPEN` / `CIRCUIT_HALF_OPEN`). O readiness inclui só `readinessState` e `db` , então o circuito aberto não tira a task do ALB.

Eventos já publicados saem da outbox no cron `OUTBOX_LIMPEZA_CRON` (padrão 04:15 UTC) depois de `OUTBOX_RETENCAO` (padrão 7 dias).

Cliente HTTP da AWS

Os dois serviços usam o `url-connection-client` do AWS SDK e excluem o `apache5-client` . O SDK 2.57.12 traz `httpcore5` 5.3.6, afetado por CVE-2026-54399 e CVE-2026-54428. Tirar o cliente Apache tira a biblioteca do classpath. Um override para `httpcore5` 5.4.4 corrigiria o CVE e deixaria a dependência, sujeita ao próximo BOM. A publicação SNS não faz long polling: socket timeout e tentativa ficam em 2 s. O consolidado, que faz long polling de 20 s, sobe o socket para 25 s. O mesmo cliente JDK serve aos dois; o timeout acompanha a chamada.

Banco, segredo e réplica

Sem `DB_SECRET_ARN` , o pool usa o driver PostgreSQL e `DB_USERNAME` / `DB_PASSWORD` . O Flyway usa esse mesmo pool de escrita.

O contrato com o Terraform é `DB_URL=jdbc:postgresql://...` . O serviço também aceita `jdbc:aws-wrapper:postgresql://` e não duplica o prefixo. No perfil `aws` , se `DB_SECRET_ARN` estiver definido, a URL efetiva é `jdbc:aws-wrapper:postgresql://...` . Na instância os plugins são `failover,awsSecretsManager` . Se o host contiver `.proxy-` (RDS Proxy), fica só `awsSecretsManager` : o failover do wrapper iria na instância e furaria o proxy. Sem segredo, o prefixo do wrapper é removido e o driver PostgreSQL usa `DB_USERNAME` / `DB_PASSWORD` . A rebusca do segredo acontece quando a autenticação falha depois da rotação. Hikari `max-lifetime` é 15 min, abaixo da rotação mínima de 30 dias. Cada conexão nova faz `SET statement_timeout = '30s'` .

`DB_READER_URL` é opcional. Quando existe, `@Transactional(readOnly = true)` vai para a réplica (`LazyConnectionDataSourceProxy` + roteamento). Sem a variável, leitura e escrita usam o primário. O Flyway continua no primário nos dois modos.

A listagem usa o índice `(comerciante_id, data_lancamento, criado_em DESC)` . O relay usa o índice parcial `outbox_eventos (proxima_tentativa_em, criado_em) WHERE publicado_em IS NULL` . Planos em `docs/performance.md` . `page` vai de 0 a 1000 e `size` de 1 a 100. Corpo de `/api/` acima de 8 KB responde 413. Campo desconhecido no JSON responde 400.

Como rodar localmente

Suba Postgres, LocalStack e Keycloak pelo compose em `infra/local` (outro módulo) e então:

```
export SERVER_PORT=8081
export DB_URL=jdbc:postgresql://localhost:5432/lancamentos
export DB_USERNAME=lancamentos
export DB_PASSWORD=lancamentos
export AWS_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:4566
export LANCAMENTOS_TOPIC_ARN=arn:aws:sns:us-east-1:000000000000:fluxo-caixa-lancamentos-events
export JWT_ISSUER_URI=http://localhost:8080/realms/fluxo-caixa
export JWT_JWK_SET_URI=http://localhost:8080/realms/fluxo-caixa/protocol/openid-connect/certs
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
export SPRING_PROFILES_ACTIVE=local
./mvnw spring-boot:run
```

Dentro do compose, `JWT_JWK_SET_URI` aponta para `http://keycloak:8080/...` e `JWT_ISSUER_URI` permanece o issuer do token obtido no host.

Imagem:

```
docker build -t lancamentos-service:local .
```

Testes

Java 25 e Docker (Testcontainers sobe PostgreSQL 17 e LocalStack).

```
./mvnw verify
```

O `verify` roda testes de domínio, casos de uso, API (MockMvc + JWT), ArchUnit, roteamento de leitura, Resilience4j (retry, timeout e abertura do circuito) e o fluxo POST → outbox → SNS → SQS, incluindo `traceparent` e `correlationId`. O JaCoCo exige pelo menos 80% de linhas em `domain` e `application`.

Matriz de compatibilidade

Versões resolvidas pelo BOM do Spring Boot 4.0.9, salvo onde há versão explícita no `pom.xml`.

Componente	Versão
Java	25
Spring Boot	4.0.9
Spring Framework	7.0.10
Resilience4j	2.4.0 (`resilience4j-spring-boot4`, `resilience4j-spring6`)
AspectJ	1.9.25.1 (`spring-boot-starter-aspectj`)
Jackson	3.1.7
Hibernate	7.2.25.Final
Flyway	11.14.1

Componente	Versão
Testcontainers	2.0.5
AWS SDK for Java v2	2.57.12
AWS Advanced JDBC Wrapper	4.6.0
springdoc-openapi	3.0.3
Micrometer / tracing	1.16.8 / 1.6.8
Driver PostgreSQL	42.7.14

Quando voltar ao Boot 4.1. O Resilience4j 2.4.0 declara `spring-boot-autoconfigure` 4.0.0. Boot 4.1 muda pacotes (health, starters) e o starter 2.4.0 não sobe nele. O gatilho é uma release do Resilience4j cujo `resilience4j-spring-boot4` dependa de `spring-boot-autoconfigure` 4.1.x (ou um artefato novo equivalente). Aí sobe junto o springdoc da linha 3.1+/3.2, que acompanha o Boot 4.1, e o BOM passa a escolher Framework, Jackson, Hibernate e Testcontainers. Até lá este serviço permanece em 4.0.9.

Colunas além do desenho mínimo

- `idempotencia.impressao_digital`: distingue repetição legítima de reuso da mesma chave com outro corpo.
- `outbox_eventos.proxima_tentativa_em`: agenda o backoff do relay entre tentativas.
- `outbox_eventos.trace_parent`, `trace_state`, `correlation_id`: contexto W3C e correlação capturados no POST.

FONTE: SERVICES/LANCAMENTOS-SERVICE/DOCS/PERFORMANCE.MD

Desempenho das consultas — lancamentos-service

Medido com `EXPLAIN (ANALYZE, BUFFERS)` no PostgreSQL 17 do Testcontainers, depois de `ANALYZE`. Volume da amostra: 4.000 lançamentos de um comerciante em 10 datas (cerca de 400 no dia consultado) e, no outbox, 1.500 pendentes fora da janela mais 30 dentro da janela do relay.

O `ORDER BY criado_em ASC` da listagem percorre o índice `(comerciante_id, data_lancamento, criado_em DESC)` de trás para frente. A contagem da página usa o mesmo índice. O relay e a limpeza usam os índices parciais.

Paginação por offset fica limitada a `page <= 1000` e `size <= 100`. Keyset (`criado_em`, `id`) e particionamento mensal de `lancamentos` ficam como evolução se o offset passar a dominar o plano.

Listagem

```
Limit  (cost=0.28..18.05 rows=50 width=95) (actual time=0.016..0.037 rows=50 loops=1)
  Buffers: shared hit=10
  ->  Index Scan Backward using idx_lancamentos_comerciante_data_criado on lancamentos
(cost=0.28..142.41 rows=400 width=95) (actual time=0.015..0.031 rows=50 loops=1)
        Index Cond: ((comerciante_id = '...'::uuid) AND (data_lancamento = '2026-11-
10'::date))
        Buffers: shared hit=10
Planning Time: 0.560 ms
Execution Time: 0.055 ms
```

Contagem da página

```
Aggregate  (cost=93.38..93.39 rows=1 width=8) (actual time=0.305..0.306 rows=1 loops=1)
  Buffers: shared hit=66
  ->  Bitmap Heap Scan on lancamentos  (cost=24.38..92.38 rows=400 width=0) (actual
time=0.050..0.259 rows=400 loops=1)
        Recheck Cond: ((comerciante_id = '...'::uuid) AND (data_lancamento = '2026-11-
10'::date))
        Heap Blocks: exact=62
        Buffers: shared hit=66
        ->  Bitmap Index Scan on idx_lancamentos_comerciante_data_criado  (cost=0.00..24.28
rows=400 width=0) (actual time=0.038..0.038 rows=400 loops=1)
              Buffers: shared hit=4
Planning Time: 0.065 ms
Execution Time: 0.344 ms
```

Relay da outbox

`SELECT ... FOR UPDATE SKIP LOCKED` com faixa em `proxima_tentativa_em` (a carga de teste fica no futuro para o relay real, que usa `now()`, não publicar esses registros). 30 linhas entram no índice parcial; as outras 1.500 ficam de fora do predicado.

```
Limit  (cost=33.30..33.67 rows=30 width=207) (actual time=0.050..0.072 rows=30 loops=1)
  Buffers: shared hit=33
  ->  LockRows  (cost=33.30..33.67 rows=30 width=207) (actual time=0.050..0.069 rows=30
loops=1)
        ->  Sort  (cost=33.30..33.37 rows=30 width=207) (actual time=0.041..0.043 rows=30
loops=1)
              Sort Key: criado_em
              Sort Method: quicksort  Memory: 28kB
              ->  Bitmap Heap Scan on outbox_eventos  (cost=4.58..32.56 rows=30 width=207)
(actual time=0.017..0.026 rows=30 loops=1)
                    Recheck Cond: ((proxima_tentativa_em >= ...) AND (proxima_tentativa_em <=
...) AND (publicado_em IS NULL))
                    Heap Blocks: exact=1
                    ->  Bitmap Index Scan on idx_outbox_eventos_pendentes  (cost=0.00..4.58
rows=30 width=0) (actual time=0.007..0.007 rows=30 loops=1)
Planning Time: 0.419 ms
Execution Time: 0.104 ms
```

O sort de 30 linhas cabe em memória. Com a janela do relay (`proxima_tentativa_em <= now()`) o mesmo índice parcial atende o filtro.

Limpeza dos publicados

```
Delete on outbox_eventos  (cost=0.13..8.15 rows=0 width=0) (actual time=0.041..0.042 rows=0
loops=1)
  ->  Index Scan using idx_outbox_eventos_publicados on outbox_eventos  (cost=0.13..8.15
rows=1 width=6) (actual time=0.040..0.040 rows=0 loops=1)
        Index Cond: ((publicado_em IS NOT NULL) AND (publicado_em < ...))
Planning Time: 0.076 ms
Execution Time: 0.066 ms
```

O que saiu do esquema

`idx_lancamentos_comerciante_data` era prefixo de `idx_lancamentos_comerciante_data_criado` e foi removido na V3. O índice parcial dos pendentes da V1 foi recriado na V3 com a mesma definição.

FONTE: SERVICES/CONSOLIDADO-SERVICE/README.MD

consolidado-service

Projeção de leitura do saldo diário de um comerciante. O serviço não é a fonte da verdade dos lançamentos: consome o evento `LancamentoRegistrado` (SNS → SQS, raw delivery) e serve consultas materializadas. Se este serviço cair, o `lancamentos-service` continua gravando; as mensagens ficam retidas na fila.

Porta local: **8082**.

Responsabilidade

- Aplicar cada lançamento no saldo do dia (`comercianteId` , `dataLancamento`), de forma comutativa.
- Garantir idempotência por `eventId` .
- Responder o saldo do dia, com saldo acumulado, e o relatório de um período de até 366 dias.
- Segurar 50 req/s com cache Redis e pool de conexões, degradando para o PostgreSQL se o Redis sair.

Arquitetura

Hexagonal, pacote base `br.com.fluxocaixa.consolidado` :

```
domain/                 agregado SaldoDiario, VO Dinheiro, relatório — sem Spring
application/            casos de uso e ports
adapters/in/web/        REST, ProblemDetail, comerciante do JWT, correlation id
adapters/in/messaging/  consumidor SQS e camada anticorrupção do evento
adapters/out/persistence/  JDBC + Flyway
adapters/out/cache/     Redis com circuito e timeout
observabilidade/        correlation id e span CONSUMER
config/                 segurança, datasource, SQS
```

O domínio não depende de Spring nem de adaptadores. A aplicação não depende de adaptadores. ArchUnit cobre essas duas regras.

Compatibilidade

Matriz presa ao que o serviço compila e testa. O Resilience4j 2.4.0 publica `resilience4j-spring-boot4` , que depende de `spring-boot-autoconfigure` 4.0.0 e de `resilience4j-spring6` . Por isso o serviço fica no Spring Boot **4.0.9** (última 4.0.x), com Java 25. O starter de AOP do Boot 4.0 é `spring-boot-starter-aspectj` .

Componente	Versão
Java	25
Spring Boot	4.0.9
Spring Framework	7.0.10

Componente	Versão
Resilience4j	2.4.0 (`resilience4j-spring-boot4`)
Jackson	3.1.7 (`tools.jackson`)
Hibernate ORM (gerido pelo Boot; o serviço usa JDBC)	7.2.25.Final
Driver JDBC PostgreSQL	42.7.14
AWS JDBC Wrapper	4.6.0
Lettuce	6.8.2.RELEASE
Testcontainers	2.0.5
AWS SDK for Java v2	2.57.12
Flyway	11.14.1
springdoc-openapi	3.0.3 (linha 3.0, construída sobre Boot 4.0)
Micrometer / Micrometer Tracing	1.16.8 / 1.6.8

Gatilho para o Boot 4.1: subir quando o Resilience4j publicar um artefato compilado e testado contra `spring-boot-autoconfigure` 4.1.x (starter novo ou release que deixe de fixar o autoconfigure 4.0.0). No mesmo movimento, trocar o springdoc para a linha 3.1+ (a 3.1.0 já nasce em cima do Boot 4.1). Até lá, permanecer na última 4.0.x.

Concorrência e idempotência

A aplicação do evento é uma soma. A ordem de chegada não muda o saldo. Por isso o incremento é um UPSERT atômico, e não optimistic locking com retry:

```
INSERT INTO saldos_diarios (...)
VALUES (...)
ON CONFLICT (comerciante_id, data) DO UPDATE SET
    total_creditos = saldos_diarios.total_creditos + EXCLUDED.total_creditos,
    total_debitos = saldos_diarios.total_debitos + EXCLUDED.total_debitos,
    quantidade_lancamentos = saldos_diarios.quantidade_lancamentos +
        EXCLUDED.quantidade_lancamentos,
    versao = saldos_diarios.versao + 1,
    atualizado_em = EXCLUDED.atualizado_em
```

A versão continua sendo gravada (1 no primeiro evento, +1 nos seguintes) para auditoria. Ela não decide o commit.

A idempotência entra na mesma transação, antes do UPSERT:

```
INSERT INTO eventos_processados (event_id, processado_em)
VALUES (?, ?)
ON CONFLICT (event_id) DO NOTHING
```

Se duas entregas do mesmo `eventId` disputam a chave, o PostgreSQL segura a segunda até o commit da primeira. A perdedora recebe 0 linhas, não mexe no saldo e a mensagem é apagada da fila. Eventos diferentes do mesmo dia somam sem lost update. Uma nova tentativa do Resilience4j depois de uma falha transitória repete essa mesma transação: ou reaplica o que não commitou, ou encontra o `eventId` e segue como duplicata.

A mensagem SQS só é apagada depois do commit. Falha transitória, esgotamento do retry ou bulkhead cheio não apagam: a fila reentrega e, após 5 recebimentos, o redrive da infra manda para `consolidado-lancamentos-dlq`. JSON inválido, corpo acima de 8 KiB, `eventType` diferente de `LancamentoRegistrado` ou `eventVersion` diferente de 1 é poison: log de erro sem o corpo e a mensagem também segue para a DLQ.

O consumidor faz long polling de 20s, pede `MessageAttributeNames=All` e `MessageSystemAttributeNames=All`, lotes de até 10 e processa o lote em virtual threads. No desligamento ele para primeiro (`SmartLifecycle` na fase máxima), termina o lote já recebido e só então o restante da aplicação cai.

O cliente HTTP é o `url-connection-client`, o mesmo do lançamentos, sem `apache5-client` e sem `httpcore5` (CVE-2026-54399 e CVE-2026-54428 no 5.3.6 que o SDK 2.57.12 traz). O `HttpURLConnection` encerra o socket no timeout. Com long polling de 20 s, o socket timeout é 25 s, a tentativa da API é 32 s e a chamada é 35 s. Abaixo de 20 s o poll morre antes da fila responder.

Resiliência

Resilience4j 2.4.0, métricas no Micrometer/Prometheus (`resilience4j_circuitbreaker_*`, `resilience4j_retry_*`, `resilience4j_bulkhead_*`, `resilience4j_ratelimiter_*`).

Peça	Nome	Comportamento
CircuitBreaker + TimeLimiter	`redis`	Timeout de 300 ms. Falhas abrem o circuito. Leitura e invalidação caem no PostgreSQL. `allowHealthIndicatorToFail=false`: estado OPEN aparece em `/actuator/health` como `CIRCUIT_OPEN` e não tira o readiness do ar
Bulkhead (semaphore)	`sqs-consumo`	Até 8 processamentos simultâneos, espera de 2 s. Se não houver permissão, a mensagem fica na fila
Retry	`banco-consumo`	3 tentativas, backoff exponencial com jitter, só para falha transitória de banco/conexão. Não retenta poison. Depois de esgotar, a mensagem não é apagada
RateLimiter	`consultas`	50 req/s. **Desligado** por padrão (`APP_RESILIENCE_RATE_LIMITER_HABILITADO=false`). Ligado, responde 429 em `application/problem+json`

O readiness inclui só `readinessState` e `db`. Redis e o circuito ficam de fora. Liveness é só `livenessState`.

Cache

Cache-aside no Redis, TTL de 30s. A chave de dados inclui uma versão por comerciante (`consolidado:versao:{id}`):

`consolidado:dia:{comerciante}:{versao}:{data}` `consolidado:relatorio:{comerciante}:{versao}:{inicio}:{fim}`

Depois do commit, a versão é incrementada. A leitura seguinte não reutiliza o valor antigo. Chaves órfãs expiram no TTL. Se o incremento falhar ou o circuito abrir, a resposta sai do banco. Consistência eventual, janela máxima de 30s quando só a invalidação falha; no caminho feliz a próxima leitura já vê o banco.

O pool Lettuce tem timeout de comando e de conexão de 300 ms. O pool Hikari de escrita tem 10 conexões (tarefa de 1 vCPU; 10 tasks ficam perto de 100 conexões no primário), timeout de conexão de 3 s, `max-lifetime` de 15 min (abaixo da rotação de 30 dias do Secrets Manager) e `statement_timeout` de 3 s em cada conexão nova.

Banco: segredo e réplica

- Sem `DB_SECRET_ARN`: driver `org.postgresql.Driver` com `DB_USERNAME` e `DB_PASSWORD`. Se a URL vier com `jdbc:aws-wrapper:`, o prefixo é removido.
- Com `DB_SECRET_ARN`: a URL efetiva é um único `jdbc:aws-wrapper:postgresql://...`, driver `software.amazon.jdbc.Driver`. Plugins `failover,awsSecretsManager` na instância. No host com `.proxy-` (RDS Proxy), só `awsSecretsManager`. Usuário e senha saem do segredo. Se `AWS_ENDPOINT_URL` estiver preenchido (local), o wrapper usa esse endpoint do Secrets Manager. O Terraform envia `jdbc:postgresql://`.
- `DB_READER_URL` opcional. Quando presente, `@Transactional(readOnly = true)` (consultas da API) vai para a réplica via `LazyConnectionDataSourceProxy` + routing datasource. O Flyway e o consumidor usam o primário. Sem a variável, há um único `DataSource`.

Rastreabilidade

- HTTP propaga W3C Trace Context (`traceparent` / `tracestate`) pelo Micrometer Tracing com bridge OpenTelemetry. `management.tracing.propagation.type=w3c`.
- Header `X-Correlation-Id`: UUID. Se ausente ou inválido, o serviço gera outro, devolve no response e coloca `correlationId` no MDC. Valor inválido não é ecoado.
- Logs JSON (formato logstash) incluem `traceId`, `spanId`, `correlationId` e, no consumo, `eventId` e `comercianteId`.
- O consumidor extrai `traceparent`, `tracestate` e `correlationId` dos message attributes (raw delivery) e abre um span `CONSUMER` filho do produtor. Mensagem sem atributos de trace inicia um trace novo. O MDC é limpo no fim de cada mensagem, na mesma virtual thread.

API

Base `/api/v1`. Erros em `application/problem+json` (RFC 9457). OpenAPI em `/v3/api-docs` e `/swagger-ui.html`. No perfil `local` o Swagger é público. No perfil `aws` o springdoc fica desligado e essas rotas não são públicas — o mesmo contrato do lançamentos. A borda não encaminha `/swagger-ui` nem `/actuator`.

O `comercianteId` sai do JWT, claim `comerciante_id` (`app.security.comerciante-claim`). Escopo `consolidado:read`, com prefixo `app.security.scope-prefix` (vazio no local, `fluxo-caixa/` na AWS). A autoridade fica `SCOPE_<prefixo>consolidado:read`.

CORS fica desligado enquanto `APP_SECURITY_CORS_ALLOWED_ORIGINS` não lista origens. Com a lista, só `GET` / `OPTIONS`, headers `Authorization`, `Content-Type`, `X-Correlation-Id`, `traceparent` e `tracestate`.

Headers de segurança na resposta: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, CSP `default-src 'none'`, `Permissions-Policy` e HSTS.

Dia sem movimento responde 200. Créditos, débitos, saldo e quantidade vêm zerados. `saldoAcumulado` continua sendo a soma dos saldos de todos os dias `<= data`, inclusive os anteriores. `atualizadoEm` fica nulo quando o dia não tem linha.

O relatório do período:

```
{
  "inicio": "2026-11-10",
  "fim": "2026-11-12",
  "dias": [
    {
      "data": "2026-11-10",
      "totalCreditos": 250.00,
      "totalDebitos": 50.00,
      "saldo": 200.00,
      "saldoAcumulado": 200.00,
      "quantidadeLancamentos": 3,
      "atualizadoEm": "2026-11-10T18:00:05Z"
    }
  ]
}
```

Um dia do intervalo sem lançamento entra na lista com zeros e com o acumulado correto. O intervalo máximo é 366 dias, inclusive. Datas fora de `YYYY-MM-DD` respondem 400 sem repetir o valor recebido.

```
curl -s http://localhost:8082/api/v1/consolidados/diario/2026-11-10 \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Correlation-Id: 11111111-1111-4111-8111-111111111111"

curl -s "http://localhost:8082/api/v1/consolidados/diario?inicio=2026-11-01&fim=2026-11-10" \
  -H "Authorization: Bearer $TOKEN"
```

Actuator liberado sem token: `/actuator/health`, `/actuator/health/liveness`, `/actuator/health/readiness`, `/actuator/info`, `/actuator/prometheus`. O ALB deve usar readiness.

Variáveis

Variável	Uso
`SERVER_PORT`	Porta HTTP. Padrão 8082

Variável	Uso
`DB_URL`	`jdbc:postgresql://` no contrato com o Terraform. Com segredo, o serviço aplica um único prefixo `jdbc:aws-wrapper:` . Os dois formatos são aceitos
`DB_USERNAME` / `DB_PASSWORD`	Credenciais do banco quando `DB_SECRET_ARN` está ausente
`DB_SECRET_ARN`	ARN do segredo do RDS. Liga o wrapper. Failover só fora do endpoint do proxy
`DB_READER_URL`	JDBC opcional da réplica. Roteia transação somente leitura
`AWS_REGION`	Padrão `us-east-1` . Também região do Secrets Manager
`AWS_ENDPOINT_URL`	LocalStack. Vazio na AWS. No profile `local` , o padrão é `http://localhost:4566`
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Com endpoint override, o padrão local é `test` / `test`
`CONSOLIDADO_QUEUE_URL`	Fila `consolidado-lancamentos-queue` . Obrigatória com o consumo ligado
`JWT_ISSUER_URI`	Emissor do JWT
`JWT_JWK_SET_URI`	Opcional. Quando preenchida, as chaves vêm dessa URL e o `iss` continua sendo validado à parte
`APP_SECURITY_SCOPE_PREFIX`	Prefixo do escopo. Vazio no local; `fluxo-caixa/` na AWS
`APP_SECURITY_COMERCIANTE_CLAIM`	Claim do comerciante. Padrão `comerciante_id`
`APP_SECURITY_CORS_ALLOWED_ORIGINS`	Origens CORS separadas por vírgula. Vazio desliga CORS
`APP_RESILIENCE_RATE_LIMITER_HABILITADO`	`true` liga o limitador das consultas. Padrão `false`
`REDIS_HOST` / `REDIS_PORT` / `REDIS_SSL`	Redis. Padrão `localhost` / `6379` / `false`
`OTEL_EXPORTER_OTLP_ENDPOINT`	Base OTLP. O Boot acrescenta `/v1/traces`
`SPRING_PROFILES_ACTIVE`	`local` ou `aws`
`APP_MESSAGING_CONSUMO_HABILITADO`	Desliga o poller. Os testes de API usam `false`

Profiles: `local` aponta o endpoint AWS para o LocalStack e abre o Swagger. `aws` não define endpoint, usa o prefixo de escopo do Cognito e autentica o Swagger.

Observabilidade

- Logs JSON no formato logstash, com MDC (`traceId` , `spanId` , `correlationId` , `eventId` , `comercianteId`).

- Métricas de domínio: `consolidado.eventos.processados`, `consolidado.eventos.duplicados`, `consolidado.eventos.falhas` e o timer `consolidado.eventos.lag`.
- Traces OTLP. Métricas OTLP ficam desligadas; o scrape é o Prometheus.
- Virtual threads ligadas (`spring.threads.virtual.enabled`).

Planos de execução das consultas críticas: docs/performance.md.

Como rodar

Pré-requisitos locais: Java 25, Docker, e o compose da infra (Postgres, Redis, LocalStack, Keycloak) sobe por outro componente.

```
cd services/consolidado-service
export SPRING_PROFILES_ACTIVE=local
export DB_URL=jdbc:postgresql://localhost:5432/consolidado
export DB_USERNAME=consolidado
export DB_PASSWORD=consolidado
export AWS_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export CONSOLIDADO_QUEUE_URL=http://localhost:4566/000000000000/consolidado-lancamentos-queue
export JWT_ISSUER_URI=http://localhost:8080/realms/fluxo-caixa
export JWT_JWK_SET_URI=http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs
export REDIS_HOST=localhost
export REDIS_PORT=6379
export REDIS_SSL=false
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
./mvnw spring-boot:run
```

Imagem (usuário não-root `app`, com `apt-get upgrade` na build):

```
docker build -t consolidado-service:1.0.0 services/consolidado-service
```

Testes

```
./mvnw verify
```

Sobe PostgreSQL 17, Redis 7 e LocalStack (SQS e, no teste do segredo, Secrets Manager) via Testcontainers. Cobre saldo, idempotência, estorno, isolamento entre comerciantes, traceparent conhecido no log JSON, circuito aberto sem derrubar readiness, seleção do driver e Flyway com o wrapper, e roteamento leitura/escrita.

JaCoCo exige 80% de linhas em `domain` e `application`.

FONTE: SERVICES/CONSOLIDADO-SERVICE/DOCS/PERFORMANCE.MD

Desempenho das consultas do consolidado

Medido em PostgreSQL 17 (imagem `postgres:17-alpine`), cache de buffers quente, sem concorrência. Volume sintético: **20 comerciantes × 1.097 dias** (2023-11-10 a 2026-11-10) = **21.940 linhas**. O script reproduzível está em explain-saldos.sql. Os tempos absolutos são de um container local; o que vale levar para a RDS é o formato do plano.

A PK `(comerciante_id, data)` é o único índice. As duas consultas críticas usam essa chave. Não há N+1: o dia é uma agregação e o período é uma sentença.

Saldo do dia e acumulado

Uma varredura `comerciante_id = ? AND data <= ?`, com `FILTER` para o movimento do dia e `SUM` para o acumulado.

```
Aggregate  (actual time=0.591..0.592 rows=1)
  -> Bitmap Heap Scan on saldos_diarios  (actual time=0.068..0.183 rows=1097)
       Heap Blocks: exact=12
       -> Bitmap Index Scan on pk_saldos_diarios  (actual time=0.058..0.058 rows=1097)
            Index Cond: (comerciante_id = ? AND data <= ?)
Planning Time: 0.643 ms
Execution Time: 0.674 ms
Buffers: shared hit=19
```

1.097 dias de um comerciante, 19 páginas em cache, **0,67 ms**.

Relatório do período

Uma sentença: soma anterior (`data < inicio`) na PK, dias do intervalo na PK, `generate_series` para as lacunas e `SUM(...) OVER (ORDER BY data)` para o saldo acumulado. Período de 31 dias:

```
WindowAgg  (actual time=1.315..1.388 rows=31)
  -> Nested Loop
       -> Merge Left Join
            -> Index Scan using pk_saldos_diarios
                 Index Cond: (comerciante_id = ? AND data >= ? AND data <= ?)
                 (actual time=0.025..0.032 rows=31)
       -> Aggregate  (actual time=1.191..1.193 rows=1 loops=1)
            -> Bitmap Index Scan on pk_saldos_diarios
                 Index Cond: (comerciante_id = ? AND data < ?)
                 (rows=1066)
Planning Time: 0.460 ms
Execution Time: 1.607 ms
Buffers: shared hit=22
```

O intervalo de 31 dias é index scan direto (4 páginas). O custo dominante é a soma de tudo que veio antes (~1,2 ms para 1.066 dias). A window function em si ordena 31 linhas. **1,61 ms** no total.

Pool e timeout

Por task (consolidado em prod é 1 vCPU): Hikari de escrita com no máximo 10 conexões, `connection-timeout` 3 s, `max-lifetime` 15 min, `keepalive` 5 min. Cada conexão nova executa `SET statement_timeout = '3s'`. As consultas medidas ficam duas a três ordens de grandeza abaixo desse teto; o timeout existe para cortar varredura acidental. Com réplica (`DB_READER_URL`), o pool de leitura repete o mesmo tamanho e é `readOnly`.

10 tasks × 10 conexões de escrita ≈ 100 no primário. Um `db.t4g.medium` comporta isso com folga, inclusive se a réplica estiver ligada.

O que não fazer agora

O plano já é range scan na PK. Índice extra, partição ou saldo materializado não mudam o p95 enquanto o cache de 30 s absorve a leitura e o histórico por comerciante cabe em ~1.000 linhas.

Quando o volume crescer

- **Índice de cobertura** `ON saldos_diarios (comerciante_id, data) INCLUDE (total_creditos, total_debitos, quantidade_lancamentos, atualizado_em)` se o `Bitmap Heap Scan` passar a dominar (hoje são 12 páginas). A PK não cobre as colunas somadas, então o plano busca o heap.
- **Snapshot mensal** (`comerciante_id`, `ano_mes`, `saldo_acumulado`) quando um comerciante passar de ~10 mil dias ou o `EXPLAIN` do acumulado passar de ~10 ms. A consulta do dia soma o snapshot do mês anterior mais os dias do mês corrente, em vez do histórico inteiro.
- **Réplica de leitura** (`DB_READER_URL`) quando o primário dividir UPSERT do consumidor com o `SUM` das consultas. O roteamento já manda `@Transactional(readOnly = true)` para a réplica.
- **Particionamento** por mês não ajuda: a PK começa por `comerciante_id`, não por data. Hash de `comerciante_id` só faria sentido com dezenas de milhões de linhas.

FONTE: INFRA/LOCAL/README.MD

Ambiente local

Compose em `infra/local`. Dois perfis:

- `infra` sobe só as dependências, para rodar o serviço pela IDE ou pelo `./mvnw`.
- `completo` sobe dependências e os dois serviços, com build do Dockerfile em `services/<servico>`.

O Compose não tem como expressar "sobe tudo por padrão e um perfil menor por cima". Os alvos do `Makefile` na raiz são o caminho esperado: `make infra-up` e `make up`.

```
make infra-up
make token
make infra-down
```

`make up` falha enquanto `services/*/Dockerfile` não existir.

O que sobe

Serviço	Porta no host	Credencial
postgres 17	5432	bancos `lancamentos` e `consolidado`, usuário = senha = nome do banco. Superusuário `postgres` / `postgres`
redis 7.4	6379	sem senha
localstack	4566	tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ com `maxReceiveCount=5`, subscription com raw delivery
keycloak 26.4.7	8080	admin `admin` / `admin`. Realm `fluxo-caixa`
otel-lgtm	3000, 4317, 4318	Grafana `admin` / `admin`

Usuários do realm:

Usuário	Senha	`comerciante_id`
`comerciante1`	`comerciante1`	`11111111-1111-1111-1111-111111111111`
`comerciante2`	`comerciante2`	`22222222-2222-2222-2222-222222222222`

Client público `fluxo-caixa-app`, direct access grants. Escopos default: `lancamentos:write`, `lancamentos:read`, `consolidado:read`. O claim `comerciante_id` vai no access token.

```
infra/local/get-token.sh
infra/local/get-token.sh comerciante2 comerciante2
```

No zsh, `USERNAME=... comando` não funciona: o zsh reserva `USERNAME` para o login do sistema. Use o argumento ou `KC_USERNAME` / `KC_PASSWORD`.

Variáveis dentro do compose

Os serviços usam o hostname da rede do compose (`postgres`, `redis`, `localstack`, `keycloak`, `otel-lgtm`). O issuer continua `http://localhost:8080/realms/fluxo-caixa`, que é o `iss` do token obtido no host. O JWK vem de `JWT_JWK_SET_URI=http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs`. O Keycloak está com `KC_HOSTNAME=http://localhost:8080` e `KC_HOSTNAME_BACKCHANNEL_DYNAMIC=true`.

`AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY` valem `test`. O LocalStack exige algum valor; isso não existe na task da AWS, que usa a task role.

Na IDE, com `make infra-up`, troque os hostnames por `localhost` e a fila por `http://localhost:4566/000000000000/consolidado-lancamentos-queue`. O resto do contrato está na seção 10 do blueprint.

O script de init do Postgres roda só na primeira subida do volume. Para recriar bancos e realm: `docker compose -f infra/local/docker-compose.yml --profile infra down -v`.

Trace e correlation id no Grafana

Os serviços exportam trace OTLP para o `otel-lgtm` (Tempo) com `OTEL_PROPAGATORS=tracecontext,baggage`. O header `X-Correlation-Id` é da aplicação: volta na resposta e entra no log JSON como `correlationId`, junto com `traceId` e `spanId`.

O Alloy lê o stdout dos containers pelo socket do Docker e entrega no Loki do `otel-lgtm` (`http://otel-lgtm:3100`). No Grafana (`http://localhost:3000`, `admin` / `admin`):

- Tempo: serviço `lancamentos` ou `consolidado`.
- Loki: `{container="fluxo-caixa-lancamentos"} | json | traceId="<id>"` ou `correlationId="<uuid>"`.

O `traceId` do log é o mesmo do Tempo. Sem o Alloy, o trace continua no Tempo e o log fica só no `docker compose logs`.

FONTE: TESTS/LOAD/README.MD

Carga do consolidado

`consolidado-pico.js` segura 50 requisições por segundo no `GET /api/v1/consolidados/diario/{data}` e, em paralelo, um fluxo menor de `POST /api/v1/lancamentos`.

Thresholds:

- `http_req_failed < 0.05` — o RNF de pico. O k6 conta status 0 e `>= 400`.
- p95 do consolidado `< 100` ms — o SLO com cache. Contra um ALB distante, suba o teto: `P95_MS=300`.
- p95 do POST de lançamentos `< 500` ms.

O teste passa se a taxa de falha ficar abaixo de 5% e a latência dentro do teto. Uma falha de threshold no final é o teste reprovando o ambiente, não um erro do script. Olhe `http_req_failed` separado de `http_req_duration`: perda acima de 5% quebra o RNF mesmo com p95 bom. p95 alto com pouca perda aponta cache frio, pool de banco ou pouca task.

```
make load
# ou
k6 run tests/load/consolidado-pico.js
k6 run -e DURATION=5m -e P95_MS=300 -e POST_RATE=5 tests/load/consolidado-pico.js
```

Pelo container, no Mac o host é `host.docker.internal`:

```
docker run --rm -i \
  -e LANCAMENTOS_URL=http://host.docker.internal:8081 \
  -e CONSOLIDADO_URL=http://host.docker.internal:8082 \
  -e TOKEN_URL=http://host.docker.internal:8080/realms/fluxo-caixa/protocol/openid-
       connect/token \
  grafana/k6 run - < tests/load/consolidado-pico.js
```

No Linux, `--network host` e as URLs `localhost` bastam.

Variáveis: `LANCAMENTOS_URL`, `CONSOLIDADO_URL`, `TOKEN_URL`, `DATA_LANCAMENTO`, `DURATION`, `P95_MS`, `POST_RATE`, `USERNAME`, `PASSWORD`, `CLIENT_ID`.

Contra o ALB da AWS, o WAF limita 30000 requisições por IP a cada 5 minutos (cerca de 100 req/s). O pico de 50 req/s de uma única máquina passa. Um teste mais agressivo precisa de mais IPs ou de um limite maior em `waf_rate_limit`.

E2E

`tests/e2e/smoke.sh` pega o token, lança um crédito de 150 e um débito de 40, estorna o débito e espera o saldo do dia fechar em 150.

`tests/e2e/resiliencia.sh` lê o saldo de `2099-01-15`, para o container do consolidado, grava crédito 10 e débito 3 (os dois precisam voltar 201) e, depois de religar, espera o saldo subir 7.

```
make smoke
make resiliencia
```

Os dois precisam dos serviços no ar (`make up`) e de `jq` , `curl` e `uuidgen` .